

Machine Learning for S&P 500 Stock Price Prediction

A Comprehensive Study on Classification and Regression Tasks

Executive Summary

This project presents a comprehensive machine learning solution for S&P 500 stock price prediction, addressing two fundamental tasks in financial forecasting: price trend classification (predicting whether stock prices will rise or fall) and price value regression (predicting actual closing prices). Using historical data from five major stocks (AAPL, AMZN, MSFT, GOOGL, JPM) over a 5-year period (2013-2018), we implemented and evaluated multiple machine learning approaches including traditional methods (Random Forest, XGBoost, Gradient Boosting) with GridSearchCV hyperparameter tuning, deep learning architectures (LSTM, Transformer), and ensemble techniques.

Our analysis reveals that deep learning models (LSTM and Transformer) generally outperform traditional machine learning approaches for both tasks, though the improvements are modest due to the inherent difficulty of stock prediction. For classification, models achieved accuracies of 50-61% with F1 scores up to 0.73. For regression, LSTM models showed the best performance with significantly lower RMSE compared to traditional methods, though most models exhibited negative R^2 values, indicating the challenging nature of the task. Notable findings include extreme metrics in some cases (Recall $\approx$ 1.0 or 0) suggesting model bias, and AUC values around 0.5 indicating limited discriminative power beyond random chance.

1. Introduction and Problem Formulation

1.1 Background

The prediction of stock prices represents one of the most challenging yet rewarding applications of machine learning in finance. The efficient market hypothesis suggests that stock prices reflect all available information, making prediction inherently difficult. However, the increasing availability of high-frequency financial data and advances in machine learning techniques have opened new possibilities for identifying patterns that may not be immediately apparent to human analysts.

1.2 Problem Statement

This project addresses the following research questions:

Classification Task: Can we accurately predict the directional movement (up or down) of stock prices for the next trading day?

Regression Task: Can we predict the actual closing price of stocks with reasonable accuracy?

Model Comparison: How do traditional machine learning models with hyperparameter tuning compare with deep learning approaches for financial time-series prediction?

1.3 Dataset Description

Source: S&P 500 Stock Data (Kaggle)

Time Period: February 8, 2013 to February 7, 2018 (5 years)

Selected Stocks:

- (1) AAPL (Apple Inc.)
- (2) AMZN (Amazon.com Inc.)
- (3) MSFT (Microsoft Corporation)
- (4) GOOGL (Alphabet Inc.)
- (5) JPM (JPMorgan Chase & Co.)

Features per Stock: Date, Open price, High price, Low price, Close price, Volume, Stock Name

Dataset Statistics:

- (1) Total records: 619,040
- (2) Number of unique stocks: 505
- (3) Selected 5 representative stocks for detailed analysis

2. System Design and Methodology

2.1 Overall Architecture

Our machine learning system follows a structured pipeline:

Raw Data → Feature Engineering → Model Training → Evaluation → Ensemble → Results

Key Design Decisions:

- (1) Two-Set Partitioning: Training (80%) and Test (20%) sets for all models
- (2) GridSearchCV with 5-Fold Cross
- (3) Validation: For traditional ML hyperparameter tuning
- (4) Fixed 100 epochs training

2.2 Feature Engineering

We implemented comprehensive feature engineering to transform raw price and volume data into 48 informative predictors:

- (1) Technical Indicators:
 - ① Moving Averages: MA5, MA10, MA20, MA50, EMA5, EMA10, EMA20 -
 - ② Momentum: ROC5, ROC10, ROC20, RSI(14)

- ③ Volatility: STD5, STD10, STD20, Bollinger Bands
- ④ Trend: MACD, MACD Signal, MACD Histogram
- (2) Volume Indicators: Volume_MA5, Volume_MA10, Volume Ratio
- (3) Price-based Features: High-Low difference, Open-Close difference, Daily returns
- (4) Lagged Features:
 - ① Close price lags: 1, 2, 3, 5 days
 - ② Return lags: 1, 2, 3, 5 days
 - ③ Volume lags: 1, 2, 3, 5 days

2.3 Target Variables

Classification: Price_Direction = (Next_Close > Current_Close).astype(int) Regression:

Target = Next_Close

2.4 Data Partitioning Strategy

- (1) Traditional ML Models:
 - ① Train-Test Split: 80% training, 20% testing
 - ② GridSearchCV with 5-fold Stratified Cross-Validation on training set
 - ③ Best parameters selected based on CV F1-score
- (2) Deep Learning Models:
 - ① Train-Test Split: 80% training, 20% testing
 - ② Sequence creation with 20-day windows
 - ③ fixed 100 epochs training
 - ④ Full training data used for model fitting

3. Machine Learning Models

3.1 Traditional ML with GridSearchCV

3.1.1 Random Forest

Hyperparameter Grid:

- (1) n_estimators: [50, 100]
- (2) max_depth: [5, 10, None]
- (3) min_samples_split: [2, 5]

3.1.2 XGBoost

Hyperparameter Grid:

- (1) n_estimators: [50, 100]
- (2) max_depth: [3, 5]
- (3) learning_rate: [0.05, 0.1]

3.1.3 Gradient Boosting

Hyperparameter Grid:

- (1) n_estimators: [50, 100]
- (2) max_depth: [3, 5]
- (3) learning_rate: [0.05, 0.1]

3.2 Deep Learning Models

3.2.1 LSTM Network

Architecture:

Input (20, 48) → LSTM(64, 2 layers) → FC(64→32) → ReLU → Dropout(0.2) → FC(32→1) → Output

Configuration: Sequence length=20, Hidden size=64, Epochs=100, Batch size=32, LR=0.001

3.2.2 Transformer Network

Architecture:

Input (20, 48) → Linear Embedding(64) → Positional Encoding → Transformer Encoder(4 heads, 2 layers) → FC → Output

Configuration: d_model=64, nhead=4, Epochs=100, Batch size=32, LR=0.001

3.3 Ensemble Methods

Voting Classifier/Regressor:

- (1) Hard Voting: Majority vote
- (2) Soft Voting: Probability average (classification only)

4. Evaluation Metrics

4.1 Classification Metrics

Accuracy: Overall correctness

Precision: Reliability of positive predictions

Recall: Coverage of actual positives

F1-Score: Harmonic mean of precision and recall

AUC-ROC: Discriminative ability

4.2 Regression Metrics

RMSE: Root Mean Squared Error

MAE: Mean Absolute Error

R²: Coefficient of determination

MAPE: Mean Absolute Percentage Error

5. Results and Analysis

5.1 Classification Results

5.1.1 Summary Table

Stock	Best Model	Accuracy	Precision	Recall	F1 Score	AUC
AAPL	LSTM	0.523	0.517	0.947	0.669	0.499
AMZN	LSTM	0.586	0.581	0.992	0.733	0.548
MSFT	Transformer	0.613	0.719	0.512	0.598	0.624
GOOGL	Transformer	0.545	0.548	0.975	0.702	0.470
JPM	Gradient Boosting	0.525	0.516	0.902	0.657	0.490

5.1.2 Detailed Analysis by Stock

(1) AAPL (Apple):

- ① Best: LSTM (Accuracy: 52.3%, F1: 0.669)
- ② Issue: Transformer shows Recall=1.0 (predicts all “up”)
- ③ Traditional ML: GridSearchCV selected conservative parameters (max_depth=5)

(2) AMZN (Amazon):

- ① Best: LSTM (Accuracy: 58.6%, F1: 0.733)
- ② LSTM achieves balanced high precision (0.581) and recall (0.992)
- ③ XGBoost shows good balance (Precision: 0.566, Recall: 0.591)

(3) MSFT (Microsoft):

- ① Best: Transformer (Accuracy: 61.3%, F1: 0.598)
- ② Critical Issue: LSTM completely failed (Precision=0, Recall=0, F1=0) Indicates training collapse, model predicted all samples as “down”
- ③ Transformer shows more stable performance

(4) GOOGL (Alphabet):

- ① Best: Transformer (F1: 0.702)

- ② XGBoost: Strong recall (0.857) with reasonable precision (0.548)
- ③ LSTM and Transformer both achieve ~55% accuracy
- (5) JPM (JPMorgan):
 - ① Best: Gradient Boosting (Accuracy: 52.5%, F1: 0.657)
 - ② Notable: Traditional ML outperforms deep learning for this financial stock
 - ③ Transformer shows recall=1.0 (all “up” predictions)
 - ④ GridSearchCV Best CV F1: 0.579

5.1.3 Key Observations and Issues

- (1) Extreme Recall Values:
 - ① High Recall (>0.9): AAPL Transformer (1.0), AMZN LSTM (0.992), GOOGL Transformer (0.975), JPM Transformer (1.0)
 - ② Cause: Models learn to exploit class imbalance by predicting majority class (“up”)
 - ③ Evidence: AUC values near 0.5 indicate poor discriminative power
- (2) LSTM Failure on MSFT:
 - ① Complete failure with all metrics = 0
 - ② Model collapsed to predicting all samples as negative
 - ③ Likely due to:
 - a. No validation set for early stopping
 - b. Class imbalance,
 - c. Training instability
- (3) GridSearchCV vs Test Performance:
 - ① CV F1 scores (0.51-0.62) generally higher than test F1 scores
 - ② Indicates some overfitting despite cross-validation
 - ③ Traditional ML more stable but conservative
- (4) AUC Analysis:
 - ① Most AUC values: 0.47-0.55
 - ② Barely better than random guessing (AUC=0.5)
 - ③ Suggests models struggle to distinguish true patterns from noise

5.2 Regression Results

5.2.1 Summary Table

Stock	Best Model	RMSE	MAE	R ²	MAPE (%)
AAPL	LSTM	15.79	14.47	-1.07	8.93
AMZN	LSTM	190.39	160.12	-1.01	14.44

Stock	Best Model	RMSE	MAE	R ²	MAPE (%)
MSFT	LSTM	8.20	6.33	-0.09	7.74
GOOGL	LSTM	113.49	100.06	-1.08	9.79
JPM	LSTM	7.97	5.69	0.20	5.52

5.2.2 Detailed Analysis

(1) LSTM Performance:

- ① Consistently lowest RMSE across all stocks
- ② JPM only stock with positive R² (0.20)
- ③ MAPE ranges from 5.5% (JPM) to 14.4% (AMZN)

(2) Traditional ML Comparison:

- ① Random Forest RMSE: 9.55 (JPM) to 233.42 (AMZN)
- ② XGBoost RMSE: 10.31 (JPM) to 235.09 (AMZN)
- ③ All traditional models show negative R²
- ④ LSTM achieves 30-40% lower RMSE than traditional methods

(3) Transformer:

- ① Second-best performer
- ② Slightly worse than LSTM but close
- ③ More stable than in classification task

5.2.3 Key Observations

(1) Negative R² Values:

- ① Most models show negative R² (worse than mean prediction)
- ② Only JPM with LSTM achieved positive R² (0.20)
- ③ Indicates extreme difficulty of price level prediction

(2) Price Scale Impact:

- ① AMZN (high price): RMSE=190, MAPE=14.4%
- ② JPM (lower price): RMSE=8, MAPE=5.5%
- ③ MAPE better for comparing across price levels

(3) Deep Learning Advantage:

- ① LSTM consistently 30-40% better RMSE than traditional ML
- ② But still insufficient for practical profitability

5.3 Comparative Analysis

5.3.1 Traditional ML vs Deep Learning

Aspect	Traditional ML (GridSearchCV)	Deep Learning
Classification Accuracy	45-53%	44-61%
Regression RMSE	Higher	Lower (30-40% better)
Training Time	Fast (seconds)	Slow (minutes)
Stability	More stable	Less stable (extreme metrics)
Hyperparameter Tuning	Automated (GridSearchCV)	Fixed architecture
Validation Required	No (uses CV)	Originally yes, now removed

5.3.2 LSTM vs Transformer

(1) Classification:

- ① LSTM: Better for AMZN, AAPL
- ② Transformer: Better for MSFT, GOOGL, JPM
- ③ Both show high recall bias

(2) Regression:

- ① LSTM: Consistently best performer
- ② Transformer: Close second

6. Discussion

6.1 Why Results Are Challenging

- (1) Market Efficiency: Prices already reflect available information (EMH)
- (2) Random Walk Component: Stock prices have significant unpredictability
- (3) Non-Stationarity: Statistical properties change over time
- (4) External Factors: News, sentiment, macro events not captured

6.2 Notable Patterns and Anomalies

- (1) High Recall Bias: Deep learning models consistently predict “up” (Recall: 0.88-1.0), suggesting: Class imbalance exploitation, Training without proper validation, Model not learning true patterns
- (2) $AUC \approx 0.5$: Discriminative ability barely exceeds random chance, indicating: Feature engineering insufficient, Market randomness dominates, Models not capturing predictive signals
- (3) MSFT LSTM Failure: Complete collapse (all metrics = 0) demonstrates risks of: No validation set for monitoring, Fixed training epochs, Insufficient regularization

6.3 GridSearchCV Effectiveness

(1) Benefits Observed:

- ① Automated hyperparameter selection
- ② Cross-validation provides robustness estimate
- ③ Prevents manual tuning bias

(2) Limitations:

- ① CV scores (0.51-0.62) higher than test scores → some overfitting
- ② Computationally expensive (increased training time significantly)
- ③ Limited improvement in final test performance

7. Conclusion

7.1 Key Findings

- (1) Modest Deep Learning Advantage: LSTM and Transformer generally outperform traditional ML, but improvements are limited (accuracy 50-61% vs 45-53%)
- (2) Extreme Metrics Issue: Deep learning models without validation sets exhibit problematic behavior (Recall $\approx$ 1.0 or 0), indicating training instability
- (3) AUC Near Random: Most models show AUC $\approx$ 0.5, suggesting limited true discriminative ability
- (4) JPM Exception: Financial stock (JPM) shows unique patterns with positive R^2 in regression and competitive traditional ML performance
- (5) GridSearchCV Value: Automated hyperparameter tuning provides stability but doesn't dramatically improve performance

7.2 Practical Implications

- (1) Classification (50-61% accuracy): May provide slight edge but insufficient alone for profitability
- (2) Regression (mostly negative R^2): Extremely challenging; JPM exception suggests some stocks more predictable
- (3) Risk Management Essential: Given modest predictive power, position sizing and stop-losses critical

7.3 Limitations and Future Work

(1) Current Limitations:

- ① Two-set partitioning removes validation-based early stopping
- ② No news sentiment or alternative data
- ③ Fixed architectures without architecture search

- ④ Survivorship bias in dataset

(2) Future Improvements:

- ① Reintroduce Validation Set: For deep learning early stopping
- ② News Integration: Sentiment analysis from financial news
- ③ Advanced Architectures: Attention-LSTM hybrids, Graph Neural Networks
- ④ Online Learning: Continuous model adaptation

Appendix

<https://github.com/lynn15730713595-collab/COMP5564.git>